arXiv:2406.18820v2 [cs.DC] 28 Jun 2024

Universal Checkpointing: Efficient and Flexible Checkpointing for Large Scale Distributed Training

Xinyu Lian[†], Sam Ade Jacobs[◇], Lev Kurilenko[◇], Masahiro Tanaka[◇]
Stas Bekman[♠], Olatunji Ruwase[*◇], Minjia Zhang[*†]

University of Illinois at Urbana-Champaign[†] Microsoft[◇] StasoSphere[♠]

{lian7, minjiaz}@illinois.edu
{samjacobs, lekurile, mtanaka, olruwase}@microsoft.com
stas@stason.org

Abstract

Existing checkpointing approaches seem ill-suited for distributed training even though hardware limitations make model parallelism, i.e., sharding model state across multiple accelerators, a requirement for model scaling. Consolidating distributed model state into a single checkpoint unacceptably slows down training, and is impractical at extreme scales. Distributed checkpoints, in contrast, are tightly coupled to the model parallelism and hardware configurations of the training run, and thus unusable on different configurations. To address this problem, we propose Universal Checkpointing, a technique that enables efficient checkpoint creation while providing the flexibility of resuming on arbitrary parallelism strategy and hardware configurations. Universal Checkpointing unlocks unprecedented capabilities for large-scale training such as improved resilience to hardware failures through continued training on remaining healthy hardware, and reduced training time through opportunistic exploitation of elastic capacity.

The key insight of Universal Checkpointing is the selection of the optimal representation in each phase of the checkpointing life cycle: distributed representation for saving, and consolidated representation for loading. This is achieved using two key mechanisms. First, the universal checkpoint format, which consists of a consolidated representation of each model parameter and metadata for mapping parameter fragments into training ranks of arbitrary model-parallelism configuration. Second, the universal checkpoint language, a simple but powerful specification language for converting distributed checkpoints into the universal checkpoint format. Our evaluation demonstrates the effectiveness and generality of Universal Checkpointing on state-of-the-art model architectures and a wide range of parallelism techniques.

1 Introduction

Distributed training frameworks such as Megatron-LM [29] and DeepSpeed [27] have been widely adopted for large-scale deep learning model training. These systems provide easy-to-use interfaces that allow users to leverage advanced parallelism strategies of multi-GPUs for training massive language models, without having to worry about the underlying system techniques that manage and tune distributed training.

Although current frameworks provide various parallelism strategies for accelerating DL training, such as ZeRO-style data parallelism (ZeRO-DP) [25], tensor-slicing parallelism (TP) [29], pipeline parallelism (PP) [12; 24], and sequence parallelism (SP) [14; 21], their support for checkpointing

[*] Equal advising

assumes a static allocation of GPU resources at the beginning of training and lacks the capability to resume training with a different parallelism strategy and hardware configuration. This makes them inefficient for a critical use case: the hardware resource changes during the training process. Modern DL models such as LLMs are resource-hungry and time-consuming, trained with massive GPUs over weeks of time, e.g., GPT-4 is trained on ~25,000 GPUs over 90-100 days [13]. In those scenarios, hardware failure can happen in the middle of the training, and a training process needs to wait until the failure is resolved, before resuming the training, which would lead to substantial resource waste and prolonged training time. Another compelling scenario is continual training from a generic LLM checkpoint for specialized LLMs (e.g., code generation), often with a different GPU budget [28; 15; 4; 19]. Unfortunately, in most current frameworks, resuming training from distributed checkpoints with different parallelism strategies where those checkpoints were trained on is not supported (Figure 1), resulting in either runtime errors or inconsistent states.

In this paper, we propose a method called *Universal Checkpointing (UCP)* that enables efficient and flexible checkpointing for a broad range of parallelism strategies. It allows users to freely resume distributed training from a checkpoint with a different parallel strategy and hardware config (e.g., GPU count) the checkpoint was trained on. The main challenge in designing UCP is defining a universal checkpoint format and key operations that enable flexible transformation of a wide range of advanced parallelism strategies, such as ZeRO-style data parallelism and 3D parallelism. Moreover, UCP should not introduce high overhead that slows down the distributed training process. Finally, resuming from UCP should not compromise model quality, i.e., the training loss should remain the same as if the training were to continue with the original parallel strategy.

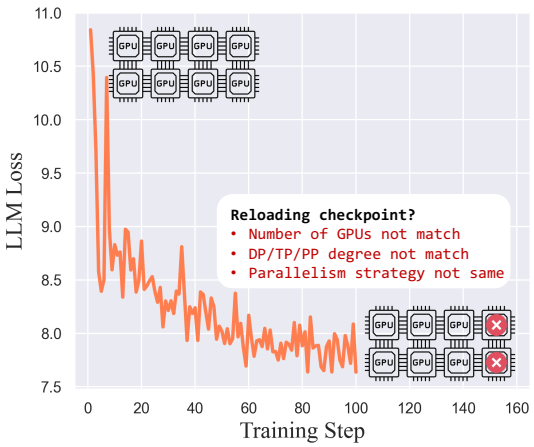


Figure 1: Current frameworks encounter challenges with GPU failures or financial budget constraints that require adjustments in GPU counts or parallelism strategies to resume training.

We have implemented UCP in DeepSpeed [27], an open-source library being used for both research and production training. UCP supports flexible checkpoint transformation along any training parallelism techniques (e.g., ZeRO-DP, TP, PP, SP). It enables elastic resource management, allowing easy scaling up and down of training and fine-tuning with varying hardware resources. UCP includes a convenient language-integrated programming interface that allows users to describe various parallelism patterns and provides transformation operations to easily transform distributed checkpoints into UCP. UCP also provides cross-framework support, enabling resuming training of checkpoints from other popular training frameworks, such as HuggingFace transformer accelerate and PyTorch lightning with DeepSpeed as a backend. We believe that UCP is the first system that enables flexible and efficient checkpointing transformation for a wide range of distributed training.

We evaluate UCP on several real-world large-scale LLM models, including Megatron-LM GPT [29], LLaMA [32], and sparse MoEs. Our evaluation results show that UCP enables system capabilities to resume training with a wide range of parallelism strategies, such as ZeRO-1/2/3 data parallelism, tensor-slicing parallelism, pipeline parallelism, and sequence parallelism, on elastic resources without compromising model quality. Our evaluation also shows that UCP is lightweight, adding zero cost when saving checkpoints and resuming training with different parallelism strategies at a small cost of UCP transformation. UCP has been used to train the BLOOM 176B model [31] and several real-world large-scale models at Microsoft such as Phi-3, greatly improving these models' resilience to hardware failures during training and reducing their training time by exploiting elastic capacity. We have open-sourced UCP as part of `https://github.com/microsoft/DeepSpeed`.

2 Background and Related Work

Checkpointing is a crucial component for DL training. DL training jobs periodically checkpoint their model parameters and optimizer states from the GPU, as well as important CPU states such as the current iteration number, to a persistent file or object store. When a training job needs to resume from a previous checkpoint, e.g. due to hardware/software failures (e.g., runtime exception from CUDA or NCCL APIs), it needs to restart all worker processes and load the previously checkpointed states from the file or object store to initialize their GPU and CPU state. The training job then proceeds to load the next minibatch of training data and resume training from the iteration after the checkpoint.

Saving and loading checkpoints for distributed training has a more involved process. For the most commonly used distributed training strategy – data parallelism, as the weights and optimizer states are replicated across GPUs, the common idiom is to have only one rank (e.g., rank 0) save the model states. The checkpoint needs to be loaded on all ranks when resuming. More advanced distributed training techniques, such as DeepSpeed ZeRO [25] and 3D parallelism [24], shard model parameters and optimizer states across GPUs, and each GPU is only responsible for checkpointing a fraction of the entire model state, creating a set of distributed checkpoints. Such a method is logically simple and also reduces the overhead of copying GPU state to host memory and persisting host memory to persistent storage. However, a limitation of this method is that it assumes the same distributed training technique is going to be used when resuming training from the distributed checkpoints. As such, existing distributed checkpointing either only supports a limited set of parallelism strategies, e.g., data distributed parallel [1], or only supports weight-only conversion for evaluation instead of continued training [8]. At the time of writing, popular training frameworks such as DeepSpeed [27], HuggingFace [34], PyTorch Lightning [10] lack support for users to easily resume training with a different parallelism strategy and hardware config the checkpoint was trained on, e.g., from 4-way tensor and 4-way pipeline model parallelism to 8-way ZeRO-style data parallelism, and vice versa.

Apart from flexible distributed checkpointing support, there are several prior work that optimizes the overhead of checkpointing for DL training. CheckFreq [23] enables more frequent checkpointing by overlapping computation with snapshot of the model states in GPU and tuning the checkpointing frequency at runtime with profiling to reduce overhead. Gemini [33] introduces in-memory checkpointing technique that checkpoints GPU state to local and remote CPUs and interleaves checkpointing IO with training computation to reduce the overhead and enable checkpointing on every iteration. Both CheckFreq and Gemini do not support advanced distributed training such as TP and PP, and their main focus is to reduce the checkpointing cost, whereas UCP enables flexible checkpointing of different distributed training techniques. Check-N-Run [9] introduces an incremental checkpointing technique that only checkpoints model states that are modified, which works well for certain types of models such as recommendation models with sparsely updated embedding tables. Different from that, UCP is generic for a wide range of distributed training techniques and large-scale DL models including transformer-based LLMs.

3 Universal Checkpointing Design and Implementation

This section first defines UCP (§ 3.1) and then introduce the UCP language (§ 3.2).

3.1 UCP Format: One Size Fits All

Distributed checkpointing challenges. One of the challenges in providing UCP is choosing a representation of data format that can be easily transformed into a wide range of parallelism techniques with varying GPUs. To see the challenge, assume we refer the parallelism technique when creating the checkpoint as *Source* and the parallelism technique we choose after resuming training from a source checkpoint as *Target*. The *Source* is often saved as distributed checkpoints when the system has more than 1 GPU, where GPU worker saves the partition of model states it owns. The rationale of this design choice is that consolidating distributed model states into a single checkpoint unacceptably slows down training and is impractical at extreme scales. When loading distributed checkpoints, each GPU loads its partition of checkpoints. However, since the distributed checkpoints saved this way are tightly coupled to the parallelism technique and hardware configurations of the training run, they become unusable on different configurations, e.g., runtime errors due to name and shape mismatches at checkpoint loading time when the number of GPUs and/or the parallelism

technique changes. In order to directly adapt the checkpoints from a *Source* to a new *Target*, we need a transformation logic. However, if there are N distributed training techniques, each with its own checkpoint saving and loading logic, it would require a total of $N \times (N-1)$ converters to support transformation from any *Source* to any *Target*, which quickly adds tons of engineering and implementation overhead.

Atom checkpoints. In a nutshell, we propose representing universal checkpoints as a set of *atom checkpoints*, which are fine-grained persistent files that consist a consolidated representation of each model parameter (e.g., *language_model.embedding.word_embeddings.weight*), as well as metadata for mapping parameter fragments into training ranks of arbitrary distributed training configurations. Without loss of generality, assuming the training uses the Adam optimizer [20], then for each parameter, UCP creates an *atom checkpoint* that includes three separate object files (e.g., .pt files in PyTorch) that correspond to:

- fp32.pt: fp32 weight values;
- exp_avg.pt: fp32 first order moment in the Adam optimizer;
- exp_avg_sq.pt: fp32 second order moment in the Adam optimizer.

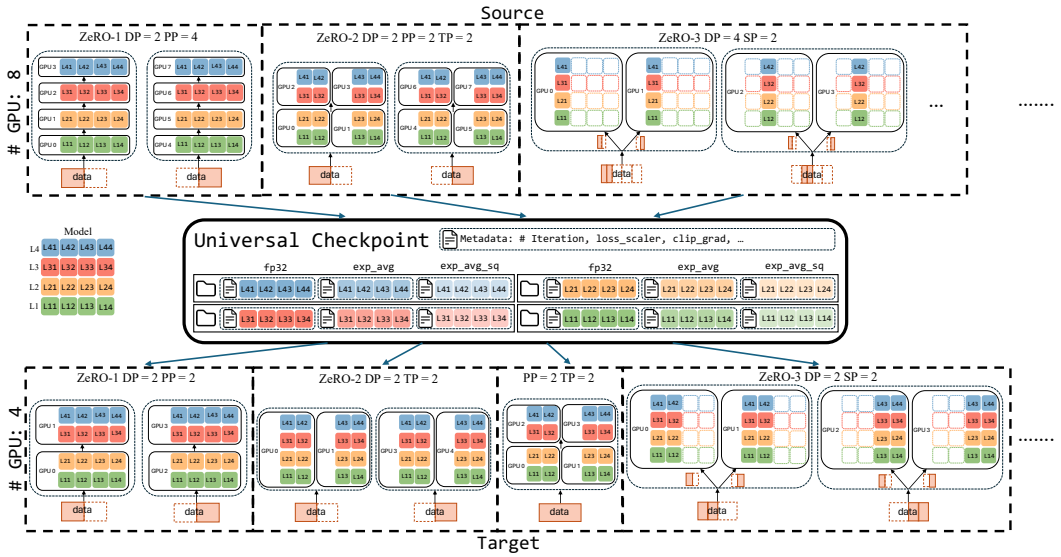


Figure 2: UCP overview. The top row illustrates training with a *Source* parallelism technique, which can be either ZeRO-DP, TP, PP, SP, or a combination of them, and how model states are partitioned across GPUs. The bottom row shows the *Target* parallelism technique, with a different hardware configuration. The middle row shows UCP , which serves as a common checkpoint format that allows any *Source* parallelism strategy to be transformed to any *Target* parallelism strategy.

This representation is useful for three reasons. First, the atomic representation of checkpoints decouples the dependencies of distributed checkpoints and specific parallelism techniques and hardware configurations. As such, one does not need to implement individual converters from each *Source* to *Target*. Instead, UCP can act as a common interchange format between different distributed training techniques, which then can be easily transformed into other distributed training strategies, as shown in Fig. 2. By keeping the consolidated representation of each model parameter, UCP enables easy split and flexible mapping of model states or fragmented states to different GPUs on a parameter-by-parameter basis, effectively reducing the working memory needed to load large model checkpoints. Second, the UCP conversion happens *lazily and on-demand*, e.g., when a training process detects a change of parallelism technique and hardware configuration. In other words, the existing distributed checkpoint saving logic does not need any change, and UCP does not introduce any additional overhead to the normal distributed training process. Third, the structure of the UCP also makes it easy to handle advanced techniques in distributed training, such as mixed-precision training. In practice, researchers and practitioners may switch between fp16 and bfloat16 mixed

precision training (MPT) [22; 18]. By keeping the fp32 weight/optimizer values, the training can resume either with fp16 or bfloat16 MPT.

3.2 UCP Language:"In-the-Box" Transformation

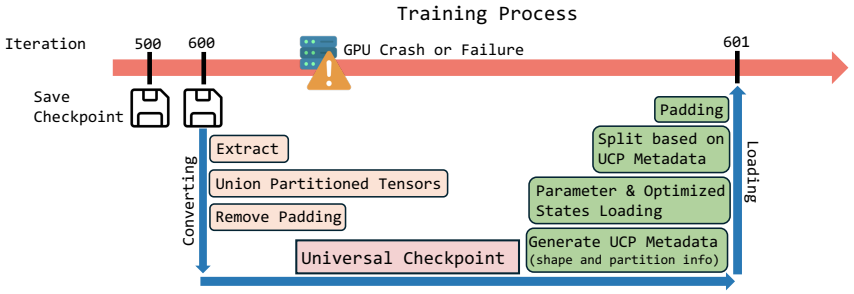


Figure 3: UCP language helps transform distributed checkpoints into the UCP format and load UCP checkpoints based on the *Target* parallel technique and new hardware configuration.

While UCP provides a common interface for different parallelism strategies, the development of transformation from arbitrary distributed checkpoints to UCP can still have a high engineering and implementation cost. This is because each GPU in distributed training calls a persist method (e.g., torch.save() in PyTorch) to save a checkpoint file of the GPU model states it owns to the disk, and the exact content of each checkpoint varies across different techniques. To tackle this challenge, UCP provides UCP language, which is a simple but powerful specification language for converting various types of distributed checkpoints into the common format described in § 3.1. UCP does this by (1) providing a declarative system with pre-defined *parameter patterns*, which cover a wide range of parallelism strategies for model states, and (2) providing a set of common operators that facilitate the transformation of distributed checkpoints into consolidated atom checkpoints. At a high-level, as illustrated in Fig. 3, UCP language is invoked when a new *Target* parallelism technique is needed or the hardware configuration changes. It first transforms distributed checkpoints into the UCP format. It then loads the UCP checkpoints based on the *Target* parallel technique and new hardware configuration.

Table 1: Parameter patterns available in UCP.

Parameter pattern	Definition
unique_params	A parameter is uniquely associated with a GPU rank.
replicated_params	A parameter is replicated across multiple GPUs.
fragment_params	A parameter partitioned along a specific dimension (e.g., row, column).
params_to_average	A parameter is updated independently across GPUs.

Table 1 presents the parameter patterns in UCP. Parameter patterns contain runtime information about how a parameter is partitioned across GPUs. For instance, *unique_params* means that a parameter is uniquely associated with a GPU rank, which is the most common pattern seen in techniques such as ZeRO-1/2 and PP. UCP language also provides more complex patterns, such as *fragment_params* , which indicates a parameter that requires partitioning along a specific dimension (e.g., row-wise and column-wise partitioning in TP). UCP also makes the list of patterns to be quite extensible to support new distributed training patterns. For example, one can easily add a pattern called *params_to_average*, which indicates that a parameter that is updated independently across GPUs.

Once identifying the pattern of a parameter, UCP provides a set of transformation operations, making it very easy to transform distributed checkpoints into the UCP format, including *Extract* , *Union* , and *StripPadding*. UCP provides *GenUcpMetadata* operation to generate new shape and location information of that can be used by a new *Target* distributed training strategy and *Load* UCP into GPU ranks. Table 2 lists and explains in more details of the main UCP operations, and Algorithm 1 demonstrates how UCP language supports consolidation of different parameter patterns based on parameter patterns and UCP operations.

Table 2: Checkpointing transformation operations available in UCP.

Operations	Meaning
Extract :	Calling *Extract* on a distributed checkpoint returns a list of parameter states contained in that checkpoint and saves each parameter state as individual checkpoint files. *Extract* can be called in parallel on multiple distributed checkpoints.
Union :	Calling the *Union* on a list of parameter states returns a list of consolidated parameters. Depending on the parameter pattern, a pattern-specific union is called on each parameter. The *Union* operation can execute in parallel at individual parameter level. More parallelism leads to faster speed but is also more memory intensive.
StripPadding :	Calling *StripPadding* strips padding from a consolidated parameter. This helps avoid saving unnecessary padding states to disk but also simplify the checkpoint loading logic.
GenUcpMetadata :	*GenUcpMetadata* calculates and generates partition metadata (e.g., shape and location information for mapping each parameter to a given rank) for each atom checkpoint based on the new *Target* strategy. Padding is also introduced when calculating the partition information.
Load :	Loads atom checkpoints to each rank based on the UCP partition metadata of *Target*. *load* leverages DeepNVMe library [26] to achieve near peak sequential read bandwidths on NVMe storage device.

Algorithm 1 Workflow of UCP Conversion

▷ Extract
for checkpoint $ckpt$ in $checkpoint_list$ **do in parallel**
  for parameter p in $ckpt$ **do**
    if `PatternMatch`(*replicated_params*, p) **then**
      continue
    else
      `Save`(p)
▷ Union
for parameter p in $parameter_list$ **do in parallel**
  $\{fp_1, fp_2, ..., fp_n\} \leftarrow$ all files name matches p
  Switch p
    case `PatternMatch`(*replicated_params*, p) **then**
      $ucp_p = fp_1$
    case `PatternMatch` (*params_to_average*, p) **then**
      ucp_p = `Sum`($fp_1, fp_2, ..., fp_n$) / n
    case `PatternMatch`(*fragment_params*, p) **then**
      ucp_p = `Concat`($fp_1, fp_2, ..., fp_n$)
    case `PatternMatch`(*unique_params*, p) **then**
      `assert`($n = 1$)
      $ucp_p = fp_1$
  if *hasPadding*(p) **then**
    ucp_p = `StripPadding`(ucp_p)
  `Save`(ucp_p)

ZeRO Stage 3 ZeRO-3 [25] fully shards model weights and optimizer states. When saving ZeRO-3 checkpoints, each DP rank persists the sharded parameters and optimizer states it owns to a checkpoint. The process of applying UCP to ZeRO-3 is illustrated in Fig. 4. It starts by using UCP language to identify parameter patterns of ZeRO-3: a parameter in a ZeRO-3 distributed checkpoint contains parameter and optimizer state fragments (*fragment_params*). Based on the pattern, UCP runs *Extract* and *Union* on fragmented parameters to create atom checkpoints, which contain consolidated parameters and their optimizer states. UCP removes padding added for hardware alignments and saves atom checkpoints to persistent files. When resuming training with ZeRO-3, each GPU calculates their new partition metadata via *GenUcpMetadata* and then loads parameter fragments and optimizer states sequentially following the layer order, with hardware alignment padding added for high performance. Once all the partitioned states are loaded into a GPU, e.g., in the flatten memory attribute *fp32_partitioned_groups_flat* of ZeRO-3, the updated attribute is then broadcast to other necessary attributes, such as *fp16_partitioned_groups_flat* for MPT.

what's this?!

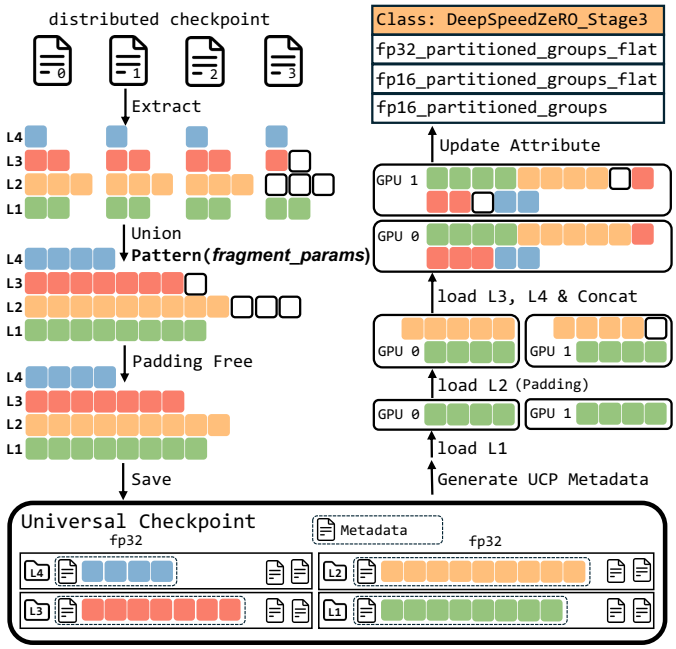


Figure 4: Workflow of UCP for ZeRO-3. The left side shows the process of converting checkpoints from *Source* parallelism (DP=4, with 4 GPUs). The right side shows how UCP checkpoints are loaded for *Target* parallelism (DP=2, with 2 GPUs).

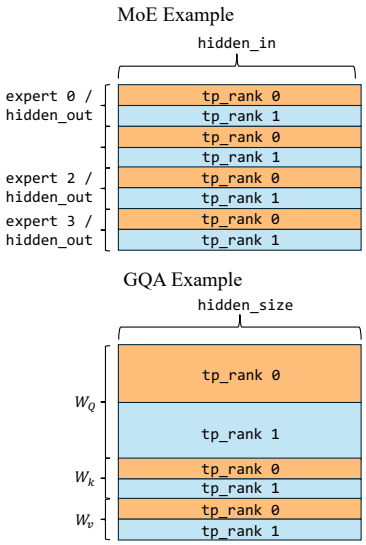


Figure 5: The sub-patterns of *fragment_params* allows UCP to identify partitions along different dimensions and variable-size fragments. This figure shows examples of MoE with 4 experts and GQA, both with TP=2.

3D parallelism 3D parallelism [24] is a common distributed training strategy that slices a model both horizontally via pipeline parallelism [29] and vertically via tensor-slicing parallelism [12]. When saving distributed checkpoints for 3D parallelism, each GPU only saves a slice of the model state it owns. UCP language identifies the parameter pattern of 3D parallelism: parameters can have *replicated_params*, *fragment_params*, *params_to_average* pattern with TP degree > 1, and *replicated_params* pattern with PP degree > 1. UCP then runs *Extract* and *Union* to consolidate each parameter based on their identified pattern to create a consolidated atom checkpoint with padding removed. For example, TP employs both row and column parallelism, sharding parameters across different dimensions. Therefore, in the union phase, these partitioned parameters need to be concatenated into a single tensor with specific dimensions. To resume 3D parallelism from UCP , a new mapping between atom checkpoints and GPU ranks is generated first, and each rank loads from atom checkpoints based on the new mapping policy.

Note that some of these patterns, such as *fragment_params*, contain sub-patterns with additional shape and partition dimension information to handle more complex checkpoints. Fig. 5 shows two examples. The MoE model in this example defines the weight tensor of an MoE's FFN layer as *[n_experts* $\times$ *hidden_out, hidden_in]* and applies TP to this layer. In this case, a sub-pattern allows UCP to identify it as a 3-dim tensor and the partition happens along the *hidden_out* dimension. In the other example, the QKV matrices in GQA [3] have different sizes but are often represented as one tensor with a shape of *[q_size + k_size + v_size, hidden]*. If TP is applied to QKV in GQA, it partitions this tensor along the first dimension for each Q, K, and V but with different sizes. A sub-pattern allows UCP to identify these variable-size fragments and take actions accordingly. Overall, UCP is quite extensible in that it allows users to easily define new (sub)-patterns to consolidate parameters.

Need to look at MoE paper to understand terminology.

4 Evaluation

We evaluate UCP through a series of experiments on training LLMs. We focus on the decoder-only Transformers: an architecture chosen due to its state-of-the-art performance [6; 2; 16]. Some of the largest models are also decoder-based [31; 30], making flexible and efficient checkpointing especially important. Overall, our results show the following:

- UCP enables saving and loading of checkpoints with varied distributed training techniques and hardware configurations without compromising model quality.
- UCP does not introduce additional distributed checkpoint saving overhead. If no nodes fail, training with UCP does not add additional GPU hours compared to normal distributed training.
- When nodes fail, UCP transformation and checkpoint loading only add minimal overhead in comparison to the end-to-end training GPU hours.

4.1 Evaluation Methodology

Workloads For the correctness evaluation, we focus on evaluating GPT-style Transformer based models. We select several architectures from prior work: GPT-3 medium [6] ($L = 24, H = 1024, A = 16, 350M$ params), LLaMA2-7B [32] ($L = 30, H = 4096, A = 16, 7B$ params), and a variant of Mixtral-7x8B MoE [17] ($L = 32, H = 4096, A = 32, E = 8, 42B$ params), to cover different model configurations and model sizes. We use a subset of the Pile dataset [11] for training, and Appendix A.1 includes the detailed hyperparameters we use for the experiments.

Hardware We conducted our experiments on: 8xH100 80GB GPUs with 800 Gbps interconnect, and 8xA100 40GB GPUs (2 4XA100, 256GB DRAM, 10TB storage, 200Gbps interconnect).

same hardware + accessible!

4.2 UCP Correctness Analysis

UCP provides flexible checkpointing from a *Source* parallelism strategy to a different *Target* with different hardware configurations. To verify this capability, we conduct correctness tests of UCP with two groups of experiments.

Single *Source* to multiple *Target* To test if UCP allows resuming training with different parallelism strategies and hardware configuration, we first train the GPT-3 model using a configuration of TP=2, PP=2, DP=2 (ZeRO-1), and SP=1. Due to constraints in time and resources, we limited the experiment to the first 200 iterations. We convert the checkpoints saved at the 100th iteration to UCP checkpoints and resume training with these UCP checkpoints using different GPU counts and parallelism strategies. We record the LM loss (average losses across the data parallel group) for each iteration. Fig. 6 illustrates that the training can be seamlessly resumed with UCP checkpoints using different *Target* parallelism strategies, achieving consistent convergence if the training were to continue with the *Source* strategy. Table 3 details the losses at iteration 101 (upon loading UCP checkpoints) and 200 (the last iteration). The difference in LM loss compared to the initial training (illustrated by the gray line) is within 0.02 difference, which is expected because GPUs introduce stochasticity from random floating-point accumulation ordering, which causes slightly inconsistent outputs between multiple runs as a result of truncating the fractional part of floating point numbers in the accumulation process [7]. These results confirm that UCP enables resuming training to different hardware and parallelism configurations.

Multiple *Source* to single *Target* Fig. 7 shows the training curves from multiple *Source* configurations to a single *Target*. Given a fixed random seed, we first train the GPT-3 model using different *Source* configurations. We then convert their distributed checkpoints saved at the 100th iteration to UCP checkpoints and resume training with a configuration of TP=2, PP=2, DP=1, and SP=1. The results show that the regardless different *Source* configurations, their checkpoints can all be converted into UCP and resume training with a different configuration. Most importantly, the resumed training curves match the curves from the *Source* at iterations 101–200. These results validate the effectiveness of UCP of converting an arbitrary configuration to a different configuration for resumed training.

Varying model architectures UCP is model architecture agnostic. As such, it is not only compatible with GPT models but also flexible enough to support various other model architectures and sizes. Fig. 8, Fig. 9, and Fig. 10 show the training convergence for LLaMA 7B [32], BLOOM 176B [31], and a variant of Mixtral-7x8B MoE [17], when resuming from UCP at the middle of training with new parallelism strategies. These figures show that training is seamlessly resumed with UCP, achieving consistent convergence that aligns with the initial training phase across these diverse models. These results suggest that UCP is quite flexible for various model architectures and sizes.

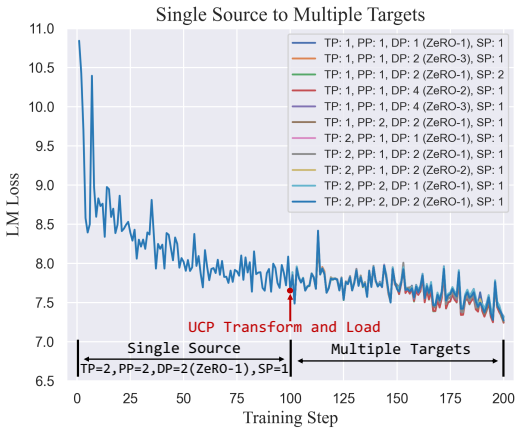


Figure 6: Training curves of loading UCP checkpoints into different *Target* at iteration 101 with various GPU counts and parallelism strategies.

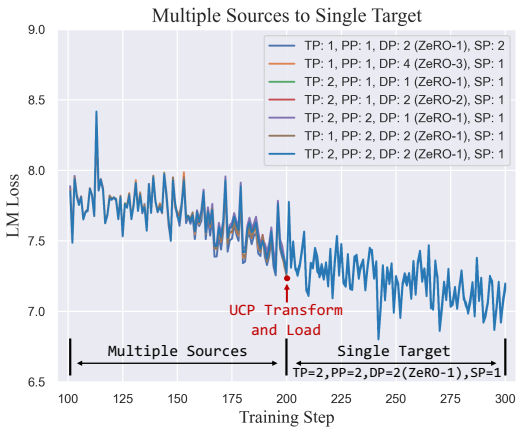


Figure 7: Training curves of transforming different *Source* parallelism strategies at iteration 100 to UCP and loading UCP a different *Target*.

Table 3: Detailed training losses at different iterations without and with UCP. The first two columns present the parallelism strategies, and the subsequent columns detail the training loss for these strategies across various iterations, when loading UCP checkpoints converted from the strategies detailed in the first row (highlighted in gray).

Target Strategy		Training Iteration					
TP/PP/DP/SP	ZeRO	loss@101	loss@120	loss@140	loss@160	loss@180	loss@200
2/2/2/1	**1**	**7.849**	**7.794**	**7.783**	**7.632**	**7.421**	**7.273**
1/1/1/1	1	7.865	7.804	7.782	7.658	7.443	7.322
1/2/2/1	1	7.865	7.801	7.778	7.648	7.408	7.256
2/1/1/1	1	7.886	7.799	7.781	7.636	7.427	7.305
1/1/2/2	1	7.870	7.791	7.772	7.605	7.342	7.243
2/1/2/1	1	7.886	7.807	7.782	7.633	7.387	7.302
2/2/1/1	1	7.887	7.794	7.772	7.632	7.420	7.287
1/1/4/1	2	7.865	7.800	7.772	7.597	7.341	7.242
2/1/2/1	2	7.887	7.799	7.776	7.643	7.423	7.303
1/1/2/1	3	7.865	7.797	7.769	7.615	7.393	7.271
1/1/4/1	3	7.862	7.801	7.780	7.620	7.381	7.268

4.3 UCP Efficiency Analysis

Saving cost UCP does not introduce additional saving costs compared to the normal distributed checkpointing. As presented in Fig. 3, the input of the UCP is the basic distributed checkpointing that is saved periodically. Therefore, the saving cost of UCP is equivalent to that of the standard training process and does not impede the training speed. To substantiate our analysis, we recorded the time required to save the checkpoints in both the standard training process and the training process with UCP enabled. Fig. 11 shows that the saving time costs are identical, confirming that UCP does not introduce any additional saving costs.

Transformation & loading cost We aim for both the conversion from distributed checkpoints to UCP and the loading of UCP checkpoints to be cost-efficient, ensuring that these processes are not significantly more expensive compared to loading standard distributed checkpoints in the standard training process. As standard distributed checkpoints cannot be loaded when there are changes in GPU counts or parallelism strategies, we keep the same GPU counts and parallelism strategies for the experiments. Fig. 12 shows that despite the additional step of conversion, the loading times for UCP checkpoints (including conversion) are only 1.14x to 1.37x compared to standard loading times. This indicates that the additional cost of using UCP, which facilitates greater flexibility in GPU counts

Is it comparable to loading with different configuration?

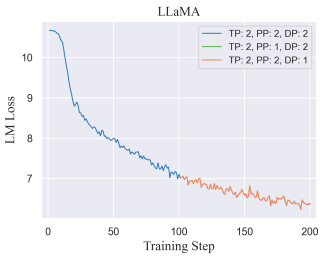


Figure 8: Training curve with LLaMA model architecture. *Source* is TP=2, PP=2, DP=2. Training is resumed at iteration 101 with new *Targets* TP=2, PP=1, DP=2 and TP=2, PP=2, DP=1.

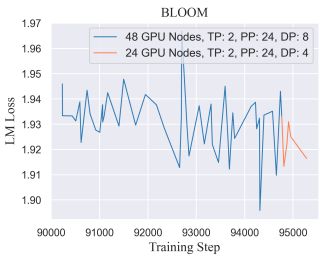


Figure 9: Training curve of BLOOM. *Source* is TP=2, PP=24, DP=8. Training is resumed at iteration 94767 with a new *Target* TP=2, PP=24, DP=4.

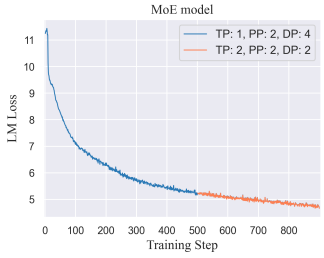


Figure 10: Training curve with a variant of the Mixtral-MoE model architecture. *Source* is TP=1, PP=2, DP=4. Training is resumed at iteration 501 with a new *Target* TP=2, PP=2, DP=2.

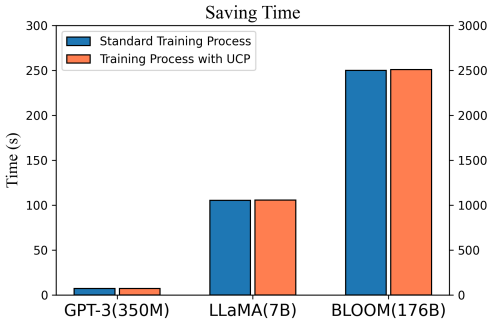


Figure 11: The time required to save distributed checkpoints for a standard training process and for a training process with UCP enabled across three models in different sizes.

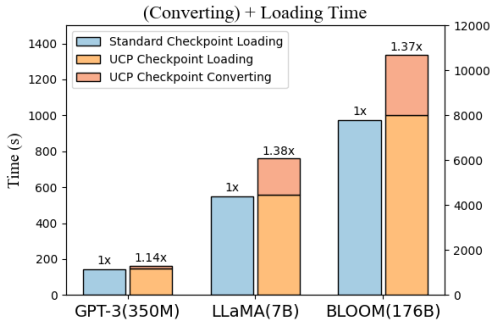


Figure 12: The time required to load normally distributed checkpoints in a standard training process, compared to the time needed to convert these distributed checkpoints into UCP checkpoints and then load the UCP checkpoints, across three models in different sizes.

and parallelism strategies, is relatively minor, given that checkpoint loading only accounts for a very small portion of the end-to-end training time.

5 Conclusion and Future Work

We have presented Universal Checkpointing (UCP), a flexible and efficient checkpointing mechanism for resuming training from distributed checkpoints with varying distributed training techniques and hardware configurations. UCP provides a common data representation for easy mapping of different distributed training strategies, including ZeRO-style data parallelism, 3D parallelism, and sequence parallelism. Meanwhile, UCP offers the UCP language, a simple yet powerful specification language for converting distributed checkpoints into UCP format. We have implemented UCP in DeepSpeed library and shown that UCP enables transforming and loading of distributed checkpoints with varying parallelism strategies and hardware configurations without affecting model convergence. In comparison to existing distributed checkpointing, UCP does not add additional checkpoint saving overhead with minimal overhead to convert and load UCP. Future work of UCP may include adding extensible patterns for emerging parallelism strategies and further improving the UCP checkpoint conversion and loading efficiency. We open-source UCP to enable new system capabilities to facilitate large-scale distributed training.

Acknowledgement

We thank Moshie Island from Habana for contributing a critical bug fix for tensor parallelism support. This work used Delta system at the National Center for Supercomputing Applications through allocation CIS240055 from the Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support (ACCESS) program. ACCESS [5] is an advanced computing and data resource program supported by the U.S. National Science Foundation (NSF) under the Office of Advanced Cyberinfrastructure awards #2138259, #2138286, #2138307, #2137603 and #2138296. The Delta advanced computing resource is a joint effort of the University of Illinois Urbana-Champaign and its National Center for Supercomputing Applications, and it is supported by the National Science Foundation (award OAC 2005572) and the State of Illinois. The work also used the Illinois Campus Cluster and NCSA NFI Hydro cluster, which are supported by the University of Illinois Urbana-Champaign and the University of Illinois System.

References

[1] PyTorch Distributed Checkpoint. https://pytorch.org/docs/stable/distributed.checkpoint.html, 2023.

[2] Meta AI. Introducing Meta LLaMA-3. https://ai.meta.com/blog/meta-llama-3/, 2024.

[3] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. {GQA:} Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, (EMNLP'23)*, December 2023.

[4] Loubna Benelbar. Santacoder Finetuning. https://github.com/loubnabnl/santacoder-finetuning, 2023.

[5] Timothy J. Boerner, Stephen Deems, Thomas R. Furlani, Shelley L. Knuth, and John Towns. ACCESS: Advancing Innovation: NSF's Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support. In *Practice and Experience in Advanced Research Computing (PEARC'23)*, 2023.

[6] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language Models are Few-Shot Learners. In *Proceedings of the 34th International Conference on Neural Information Processing Systems (NIPS'20)*, December 2020.

[7] Yuan-Hsi Chou, Christopher Ng, Shaylin Cattell, Jeremy Intan, Matthew D. Sinclair, Joseph Devietti, Timothy G. Rogers, and Tor M. Aamodt. Deterministic Atomic Buffering. In *Proceedings of the 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'20)*, October 2020.

[8] NVIDIA Corporation. Megatron-LM: Evaluation and Tasks. https://github.com/NVIDIA/Megatron-LM?tab=readme-ov-file#evaluation-and-tasks, 2024.

[9] Assaf Eisenman, Kiran Kumar Matam, Steven Ingram, Dheevatsa Mudigere, Raghuraman Krishnamoorthi, Krishnakumar Nair, Misha Smelyanskiy, and Murali Annavaram. Check-N-Run: a Checkpointing System for Training Deep Learning Recommendation Models. In *Proceedings of the 19th USENIX Symposium on Networked Systems Design and Implementation (NSDI'22)*, 2022.

[10] William Falcon and the PyTorch Lightning team. PyTorch Lightning. https://github.com/PyTorchLightning/pytorch-lightning, 2019.

[11] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, et al. The pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020.

[12] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Xu Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems (NIPS'19)*, December 2019.

[13] Stephen M. Walker II. Everything We Know About GPT-4. https://klu.ai/blog/gpt-4-llm, 2023.

[14] Sam Ade Jacobs, Masahiro Tanaka, Chengming Zhang, Minjia Zhang, Shuaiwen Leon Song, Samyam Rajbhandari, and Yuxiong He. DeepSpeed Ulysses: System Optimizations for Enabling Training of Extreme Long Sequence Transformer Models. *arXiv preprint arXiv:2309.14509*, 2023.

[15] Sami Jaghouar, Alaeddine Abdessalem, Sebastian Weisshaar, and Scott Martens. Fine-Tuning Falcon40b for Code Generation. `https://jina.ai/news/fine-tuning-falcon40b-for-code-generation`, 2023.

[16] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mistral 7B. *arXiv preprint arXiv:2310.06825*, 2023.

[17] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de Las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lélio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mixtral of Experts. *arXiv preprint arXiv:2401.04088*, 2024.

[18] Dhiraj Kalamkar, Dheevatsa Mudigere, Naveen Mellempudi, Dipankar Das, Kunal Banerjee, Sasikanth Avancha, Dharma Teja Vooturi, Nataraj Jammalamadaka, Jianyu Huang, Hector Yuen, Jiyan Yang, Jongsoo Park, Alexander Heinecke, Evangelos Georganas, Sudarshan Srinivasan, Abhisek Kundu, Misha Smelyanskiy, Bharat Kaul, and Pradeep Dubey. A Study of BFLOAT16 for Deep Learning Training. *arXiv preprint arXiv:1905.12322*, 2019.

[19] Maria Khalusova. Fine-tuning a Code LLM on a Single GPU. `https://huggingface.co/learn/cookbook/fine_tuning_code_llm_on_single_gpu`, 2023.

[20] Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR'15)*, May 2015.

[21] Vijay Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. Reducing Activation Recomputation in Large Transformer Models. *arXiv preprint arXiv:2205.05198*, 2022.

[22] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory F. Diamos, Erich Elsen, David García, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed Precision Training. In *Proceedings of the 6th International Conference on Learning Representations (ICLR'18)*, May 2018.

[23] Jayashree Mohan, Amar Phanishayee, and Vijay Chidambaram. CheckFreq: Frequent, Fine-Grained DNN Checkpointing. In *Proceedings of the 19th USENIX Conference on File and Storage Technologies (FAST'21)*, February 2021.

[24] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Anand Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. *arXiv preprint arXiv:2104.04473*, 2021.

[25] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. ZeRO: Memory Optimization Towards Training A Trillion Parameter Models. *arXiv preprint arXiv:1910.02054*, 2019.

[26] Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, and Yuxiong He. ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning. *arXiv preprint arXiv:2104.07857*, 2021.

[27] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters. In *Proceedings of the 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD'20)*, August 2020.

[28] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. Code Llama: Open Foundation Models for Code. *arXiv preprint arXiv:2308.12950*, 2024.

[29] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. *arXiv preprint arXiv:1909.08053*, 2020.

[30] Shaden Smith, Mostofa Patwary, Brandon Norick, Patrick LeGresley, Samyam Rajbhandari, Jared Casper, Zhun Liu, Shrimai Prabhumoye, George Zerveas, Vijay Korthikanti, Elton Zhang, Rewon Child, Reza Yazdani Aminabadi, Julie Bernauer, Xia Song, Mohammad Shoeybi, Yuxiong He, Michael Houston, Saurabh Tiwary, and Bryan Catanzaro. Using DeepSpeed and Megatron to Train Megatron-Turing NLG 530B, A Large-Scale Generative Language Model. *arXiv preprint arXiv:2201.11990*, 2022.

[31] BigScience team. BLOOM: A 176B-Parameter Open-Access Multilingual Language Model. *arXiv preprint arXiv:2211.05100*, 2023.

[32] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. LLaMA: Open and Efficient Foundation Language Models. *arXiv preprint arXiv:2302.13971*, 2023.

[33] Zhuang Wang, Zhen Jia, Shuai Zheng, Zhen Zhang, Xinwei Fu, T. S. Eugene Ng, and Yida Wang. GEMINI: Fast Failure Recovery in Distributed Training with In-Memory Checkpoints. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP'23)*, October 2023.

[34] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, and Jamie Brew. HuggingFace's Transformers: State-of-the-art Natural Language Processing. *arXiv preprint arXiv:1910.03771*, 2019.

A Appendix

A.1 Hyperparameters

In this part, we include detailed hyperparameters used for experiments in this work. We largely follow prior works [6; 32; 31; 17] to set the hyperparameters. Table 4 provides the detailed hyperparameters used for training models in Section 4.

Table 4: Sizes, architectures, and hyperparameters of the models in experiments.

	GPT-3	LLaMA	BLOOM	MoE
Num. parameters	350M	7B	176B	42B
Num. layers	24	32	70	32
Hidden size	1024	4096	14336	4096
Num. attention heads	16	32	112	32
Num. experts per layer	1	1	1	8
Context/sequence length	2K	2K	2K	2K
Batch size	256	256	256	256
Learning rate	3.0E-04	3.0E-04	6.0E-04	1.20E-04
Min. learning rate	3.0E-06	3.00E-06	6.0E-04	1.20E-04
Adam beta1	0.9	0.9	0.9	0.9
Adam beta2	0.95	0.95	0.95	0.95
Weight decay	0.1	0.1	0.1	0.1
Grad clip	1	1	1	1